RAMAIAH
Institute of Technology

Performance Analysis of LLMs using MongoDB Vector Search

Presented By
Afreen S - 1MS25SCS009-T
Vachana U - 1MS25SCS022-T

For the subject
Advanced Database Management Systems (MCS12)
Term: Nov 2025 - Feb 2026

Under the Guidance of
Dr. Ganeshayya Shidaganti
Associate Professor
Ramaiah Institute of Technology

TABLE OF CONTENTS

Sl No.	Chapters	Pg No.
1	Abstract	1
2	Introduction	2
3	Problem Statement	5
4	Literature Survey	7
5	Methodology	9
6	Implementation	10
7	Results	22
8	Conclusion	27
9	References	28

ABSTRACT

Large Language Models (LLMs) are widely used for tasks such as answering questions and generating text. However, their responses can become more accurate and relevant when supported with external context instead of relying only on learned patterns. This project focuses on analyzing LLM performance using MongoDB Vector Search as a semantic retrieval system.

In this approach, a collection of documents is converted into vector representations and stored in MongoDB. When a user enters a query, the system performs similarity-based retrieval to fetch the most relevant documents by meaning rather than keywords. The retrieved content is then combined into a context prompt and provided to the language model to generate an informed answer. Performance is assessed using similarity score analysis and cosine similarity measurements, including average similarity, score variation, and score drop across retrieved results.

The findings show that retrieval-based grounding helps improve response quality by reducing vague or unrelated outputs and strengthening relevance to the query. Overall, the project demonstrates that MongoDB Vector Search provides an efficient and scalable foundation for building retrieval-augmented LLM systems suitable for practical applications in intelligent search and question answering.

INTRODUCTION

Vector Search in MongoDB

Vector search is a modern information retrieval technique that enables databases to search and rank data based on semantic similarity rather than exact keyword matches. In MongoDB, vector search is implemented by storing high-dimensional numerical representations, called embeddings, within documents and performing similarity comparisons between these vectors. Embeddings are typically generated using machine learning or deep learning models that convert unstructured data such as text, images, or audio into dense numerical vectors that capture semantic meaning. Unlike traditional search, which relies on lexical matching, vector search allows MongoDB to retrieve documents that are conceptually related to a query even if they do not share the same words.

MongoDB supports vector search through Atlas Vector Search, which is built on top of the Apache Lucene engine. Each document in a MongoDB collection can store an embedding vector as an array of floating-point numbers. When a query vector is provided, MongoDB computes the similarity between the query vector and stored vectors to identify the most relevant documents. This approach is particularly useful for applications such as semantic search, recommendation systems, document similarity detection, and retrieval-augmented generation (RAG) pipelines. MongoDB's integration of vector search within a general-purpose NoSQL database allows developers to combine vector similarity queries with traditional filters, aggregations, and metadata constraints in a single query.

Scalability is a key advantage of MongoDB's vector search. As datasets grow larger and embeddings increase in dimensionality, MongoDB efficiently indexes and retrieves vectors without requiring a separate specialized vector database. This unified architecture simplifies system design and reduces operational complexity, making MongoDB suitable for enterprise-grade AI and data-intensive applications.

Distance Metrics Used in Vector Search

Distance metrics play a crucial role in vector search, as they determine how similarity between vectors is measured. One of the most commonly used metrics is cosine similarity, which

measures the cosine of the angle between two vectors rather than their absolute distance. Cosine similarity focuses on the orientation of vectors and is invariant to their magnitude, making it particularly effective for text embeddings where the direction of the vector represents semantic meaning. In MongoDB vector search, cosine similarity is widely used for natural language processing tasks such as document retrieval and semantic matching.

Another important metric is Euclidean distance, which calculates the straight-line distance between two vectors in multi-dimensional space. This metric considers both direction and magnitude, making it suitable for applications where absolute differences between vectors matter, such as image embeddings or numerical feature comparisons. However, in very high-dimensional spaces, Euclidean distance can become less effective due to the curse of dimensionality, where distances between points become increasingly similar.

In addition to cosine and Euclidean distance, other metrics such as dot product are also used in vector search systems. The dot product measures similarity by multiplying corresponding vector components and summing the results, which can be computationally efficient and is often used when vectors are normalized. MongoDB allows the selection of appropriate distance metrics depending on the nature of the data and the application requirements, enabling flexible and accurate similarity searches.

Vector Search Indexes in MongoDB

Search indexes are essential for enabling fast and efficient vector search operations in MongoDB. A vector search index organizes embedding vectors in a way that allows approximate nearest neighbor (ANN) searches to be performed efficiently, even on large datasets. MongoDB uses Lucene-based indexing techniques such as hierarchical navigable small world (HNSW) graphs to optimize vector retrieval. These structures significantly reduce the search space, allowing MongoDB to return highly relevant results with low latency.

Unlike traditional B-tree indexes used for exact matches, vector indexes are designed for similarity-based queries. MongoDB allows vector indexes to coexist with standard indexes, enabling hybrid queries that combine semantic similarity with structured filters such as date ranges, categories, or user IDs. This hybrid indexing capability is particularly valuable in

real-world applications where relevance depends on both semantic meaning and contextual constraints.

The configuration of vector search indexes includes defining the vector field, dimensionality, and distance metric. Once created, these indexes automatically update as new documents are inserted or existing documents are modified, ensuring consistency between stored data and search results.

Effect of Vector Search on Large Language Models (LLMs)

Vector search has a significant impact on the performance and applicability of large language models (LLMs), particularly in enterprise and real-world use cases. One of the most important contributions of vector search is its role in retrieval-augmented generation (RAG). In RAG architectures, vector search is used to retrieve relevant external documents or knowledge from databases like MongoDB, which are then provided as context to an LLM. This approach reduces hallucinations, improves factual accuracy, and enables LLMs to generate responses grounded in up-to-date and domain-specific information.

By integrating MongoDB vector search with LLMs, organizations can store embeddings of internal documents, policies, manuals, or codebases and retrieve them dynamically at inference time. This eliminates the need to retrain models for every knowledge update and allows LLM applications to scale efficiently. Vector search also improves response relevance by ensuring that only the most semantically similar content is passed to the model, thereby reducing token usage and inference costs.

Furthermore, vector search enables personalization and contextual understanding in LLM-powered systems. By embedding user queries, preferences, or historical interactions, MongoDB can retrieve highly relevant contextual data that enhances the LLM's ability to generate tailored and meaningful responses. As a result, vector search acts as a critical bridge between static language models and dynamic, real-world data, significantly expanding the practical capabilities of LLM-based applications.

PROBLEM STATEMENT

Large Language Models (LLMs) are widely used for generating answers, explanations, and text-based assistance. While these models can produce fluent responses, they do not always provide reliable or relevant outputs when the user query requires supporting information. In such cases, the model may generate responses based on general patterns rather than grounded content, which can reduce accuracy and lead to unclear or incomplete answers.

A key challenge in improving LLM response quality is retrieving the right context efficiently. Traditional keyword-based search methods depend heavily on exact word matches, which often fails when users phrase queries differently or when documents contain the same meaning expressed in different terms. This creates a mismatch between the user's intent and the documents selected to support the response, affecting both retrieval quality and the final generated answer.

To address this issue, semantic retrieval using vector similarity can be used to find documents based on meaning instead of keywords. In this project, MongoDB Vector Search is used to store and retrieve vector-based document representations, enabling similarity-driven retrieval of top relevant results. The project aims to analyze how effectively this retrieval method supports LLM-generated responses by examining similarity scores and cosine similarity-based measures such as average relevance, score variance, and score drop across retrieved results, thereby evaluating system consistency and performance.

Objectives

The main objective of this project is to evaluate the performance of LLM-based question answering by enhancing it with semantic retrieval using MongoDB Vector Search and analyzing the effectiveness of the retrieved context:

- To create a document dataset and store it in MongoDB for retrieval-based applications.
- To convert document text into vector representations for semantic similarity search.
- To implement MongoDB Vector Search for retrieving the most relevant documents for a given query.

- To integrate the retrieved documents as contextual input for generating responses using an LLM.
- To analyze retrieval quality using similarity scores and cosine similarity measurements.
- To evaluate the consistency of retrieval results using statistical measures such as average similarity, variance, and score drop.
- To study how retrieval-based grounding improves the relevance and usefulness of generated answers.

LITERATURE SURVEY

Research findings indicate that MongoDB Atlas Vector Search significantly improves semantic retrieval compared to traditional keyword-based search. Embedding-based similarity search enables better contextual understanding of natural language queries and enhances the relevance of retrieved documents. These improvements are especially evident in applications involving question answering systems, chatbots, and retrieval-augmented generation workflows.

Integration of vector search with large language models further strengthens response grounding and contextual coherence. By retrieving semantically relevant information prior to generation, vector search reduces hallucinations and improves factual consistency in generated outputs. MongoDB's ability to unify vector search with document storage simplifies system design and deployment.

Supporting research on dense retrieval and approximate nearest neighbor indexing confirms that vector-based retrieval is both scalable and efficient. Collectively, these findings establish vector search as a core component of modern AI-driven information retrieval systems.

Limitations

A major limitation identified across existing studies is the restricted scale of experimental evaluation. Most implementations are tested on small to medium-sized datasets, limiting insight into system behavior under large-scale, real-world workloads involving millions of vectors.

Another limitation is the limited use of standardized information retrieval metrics. Many evaluations rely on qualitative analysis rather than metrics such as recall, precision, or ranking-based measures, making objective comparison challenging.

In addition, current research provides limited exploration of index-level optimization and parameter tuning. The lack of detailed analysis on performance–accuracy trade-offs constrains deeper technical understanding.

Scope for Improvement

Future research should prioritize large-scale benchmarking to evaluate scalability, latency, and memory efficiency under realistic enterprise workloads. Such evaluations would provide stronger empirical evidence for production-level deployment.

Further improvements can be achieved by incorporating standardized retrieval metrics and conducting comparative studies against specialized vector databases. Investigating index configuration and optimization strategies would also enhance system performance and reliability.

METHODOLOGY

This project follows a retrieval-based workflow to evaluate how effectively an LLM can answer a question when it is supported with relevant context. Instead of directly generating an answer from the model alone, the system first searches for semantically related documents stored in a database and then uses those retrieved documents as supporting information. This helps reduce irrelevant responses and improves the chances of producing an answer that matches the user's intent.

To begin, a set of sample documents is created, with each document containing a title and a short content paragraph. These documents are converted into vector embeddings using a SentenceTransformer model, which transforms text into numerical representations that capture meaning. To make similarity comparisons more reliable, the embeddings are normalized using NumPy, ensuring they are on a comparable scale. Once embeddings are generated, each document is updated to include its embedding vector, and the complete dataset is stored inside MongoDB Atlas in a dedicated vector collection.

For retrieval, the system accepts a user query and generates a query embedding using the same embedding model. MongoDB Vector Search is then used to retrieve the top matching documents by applying the $vectorSearch aggregation stage. This method selects documents based on semantic similarity rather than exact keyword matches. Parameters such as numCandidates and limit are used to control how many potential matches are scanned and how many final results are returned, helping balance speed and retrieval quality.

After retrieval, the content from the top-ranked documents is combined into one context block. This context is inserted into a structured prompt along with the user query, forming a retrieval-augmented input. A text-generation model, GPT-2 via Hugging Face Transformers, then produces the final answer based on the provided context. To evaluate how well retrieval is working, cosine similarity is calculated between the query vector and the retrieved document vectors using scikit-learn. Metrics such as average similarity, similarity variance, and similarity drop from the best match to the weakest match are used to measure consistency and relevance of retrieved results. These measurements help in analyzing how strongly MongoDB Vector Search supports the LLM in producing better grounded response

IMPLEMENTATION

The code snippet demonstrates the configuration and initialization of a connection between a Python application and a MongoDB Atlas cloud-based database using the PyMongo library. It creates a MongoDB client instance, establishes a secure connection to the remote cluster, and accesses a designated database and collection for performing data storage and retrieval operations.

```
from pymongo import MongoClient
client = MongoClient(
"mongodb+srv://afreensyed862_db_user:afreensyed862_db_user@cluster0.oujgpl9.mongodb.net
/myDB?retryWrites=true&w=majority"
)
db2=client["Vector_DB"]
collection2=db2["vectors"]
```

This code implements a complete pipeline for semantic representation and persistent storage of textual data using transformer-based embeddings. A pre-trained sentence embedding model is employed to convert document text into dense numerical vectors that capture semantic meaning. These embeddings are then normalized and stored along with the original document metadata in a MongoDB collection, enabling efficient similarity-based retrieval.

The implementation begins by initializing a lightweight yet effective transformer model (all-MiniLM-L6-v2) designed for semantic similarity tasks. A diverse set of documents covering artificial intelligence concepts as well as general, unrelated topics is defined to demonstrate the model's ability to distinguish semantically similar and dissimilar content. For each document, the textual content is transformed into a fixed-length vector embedding that represents its contextual meaning in a high-dimensional space.

To ensure consistency and improve the effectiveness of similarity computations, each embedding is normalized to unit length using L2 normalization. This step is particularly important for cosine similarity–based retrieval, as it removes magnitude variations and allows similarity to be computed purely based on vector orientation. The normalized embeddings are then attached to their respective documents as additional attributes.

Finally, the enriched documents—now containing both textual information and their corresponding vector embeddings—are inserted into a MongoDB collection in a batch operation. This creates a persistent vector store that can be leveraged for downstream tasks such as semantic search, nearest-neighbor retrieval, and retrieval-augmented generation (RAG).

```
import numpy as np

from sentence_transformers import SentenceTransformer

model = SentenceTransformer("all-MiniLM-L6-v2")

docs = [

    {"title": "ML Basics", "content": "Machine learning embeddings can help find similar documents."},

    {"title": "AI & Technology", "content": "I enjoy reading books on technology and AI."},

    {"title": "AI Fields", "content": "Artificial intelligence and machine learning are fascinating fields."},

    {"title": "ML Testing", "content": "Testing machine learning models is important for accuracy and reliability."},

    {"title": "AI Applications", "content": "Applications of AI include natural language processing, computer vision, and robotics."},

    {"title": "Deep Learning", "content": "Deep learning is a subset of machine learning using neural networks for complex tasks."},

    {"title": "Reinforcement Learning", "content": "Reinforcement learning teaches agents to make decisions based on rewards and penalties."},

    {"title": "Natural Language Processing", "content": "NLP focuses on enabling machines to understand and generate human language."},

    {"title": "Computer Vision", "content": "Computer vision allows computers to interpret and analyze visual data from the world."},
```

{"title": "AI Ethics", "content": "AI ethics explores the responsible and fair use of artificial intelligence systems."},

{"title": "Supervised Learning", "content": "Supervised learning trains models using labeled datasets to predict outputs."},

{"title": "Unsupervised Learning", "content": "Unsupervised learning discovers hidden patterns in unlabeled data."},

{"title": "Neural Networks", "content": "Neural networks are inspired by the human brain and used in AI tasks."},

{"title": "Generative AI", "content": "Generative AI models can create text, images, audio, and video content."},

{"title": "AI in Healthcare", "content": "AI improves diagnostics, treatment planning, and patient monitoring."},

{"title": "AI in Finance", "content": "Finance uses AI for fraud detection, risk analysis, and automation."},

{"title": "AI in Robotics", "content": "Robotics integrates AI to enable autonomous decision-making."},

{"title": "AI Research Trends", "content": "Modern AI research focuses on efficiency, fairness, and explainability."},

{"title": "Transfer Learning", "content": "Transfer learning reuses knowledge from one task to solve another."},

{"title": "AI Model Deployment", "content": "Deployment involves serving models in production environments."},

{"title": "Computer Vision Models", "content": "Popular models include YOLO, ResNet, and Vision Transformers."},

{"title": "Language Models", "content": "Large language models excel at text understanding and generation."},

{"title": "AI in Education", "content": "AI personalizes learning experiences and automates grading."},

{"title": "AI in Transportation", "content": "Self-driving vehicles rely on AI perception and planning systems."},

{"title": "Explainable AI", "content": "Explainable AI improves transparency and trust in model decisions."},

{"title": "AI Optimization", "content": "Optimization techniques improve model accuracy and convergence speed."},

{"title": "AI in Security", "content": "AI detects cyber threats and abnormal network behavior."},

{"title": "Edge AI", "content": "Edge AI enables local inference on devices with low latency."},

{"title": "Federated Learning", "content": "Federated learning trains models without sharing raw data."},

{"title": "Self-Supervised Learning", "content": "Self-supervised learning uses unlabeled data with proxy objectives."},

{"title": "AI in NLP Tasks", "content": "AI performs summarization, translation, and sentiment analysis."},

{"title": "AI in Vision Tasks", "content": "Vision systems perform detection, segmentation, and recognition."},

{"title": "AI in Robotics Tasks", "content": "Robots use AI for navigation and manipulation."},

{"title": "AI Research Challenges", "content": "Bias, robustness, and generalization remain open challenges."},

{"title": "AI Datasets", "content": "Datasets like ImageNet and COCO are widely used in research."},

{"title": "AI Frameworks", "content": "TensorFlow and PyTorch are popular AI frameworks."},

{"title": "AI Hardware", "content": "Specialized hardware accelerates AI workloads."},

{"title": "AI Trends 2025", "content": "Trends include multimodal models and AI safety research."},

{"title": "AI Careers", "content": "AI careers span research, engineering, and operations."},

{"title": "AI Startups", "content": "Startups apply AI to healthcare, fintech, and education."},

{"title": "AI Tools", "content": "AI tools assist with labeling, training, and deployment."},

{"title": "AI Pipelines", "content": "Pipelines automate data processing and model training."},

{"title": "Model Evaluation", "content": "Evaluation measures accuracy, robustness, and generalization."},

{"title": "Embedding Models", "content": "Embedding models convert text into numerical vectors."},

{"title": "Vector Databases", "content": "Vector databases store embeddings for similarity search."},

{"title": "Semantic Search", "content": "Semantic search retrieves documents by meaning, not keywords."},

{"title": "Retrieval-Augmented Generation", "content": "RAG combines retrieval with text generation."},

{"title": "Prompt Engineering", "content": "Prompt design improves LLM response quality."},

{"title": "LLM Fine-Tuning", "content": "Fine-tuning adapts language models to specific tasks."},

{"title": "Instruction Tuning", "content": "Instruction tuning improves model alignment with user intent."},

{"title": "Cooking Pasta", "content": "Boil water, add salt, cook pasta."},

{"title": "Football Match", "content": "The team scored a goal in the final minute."},

{"title": "Mountains", "content": "Hiking in the Himalayas is challenging."},

{"title": "Baking Bread", "content": "Mix flour, yeast, and water, then bake the dough in an oven."},

{"title": "Making Tea", "content": "Boil water, add tea leaves, strain, and serve hot."},

{"title": "Frying Eggs", "content": "Heat oil in a pan and fry eggs until the yolk sets."},

{"title": "Multimodal AI", "content": "Multimodal AI processes text, images, and audio together."},

{"title": "AI Safety", "content": "AI safety focuses on preventing harmful behavior."},

{"title": "AI Alignment", "content": "Alignment ensures models follow human values."},

{"title": "Knowledge Graphs", "content": "Knowledge graphs represent structured relationships."},

{"title": "Search Engines", "content": "Modern search engines use embeddings for relevance."},

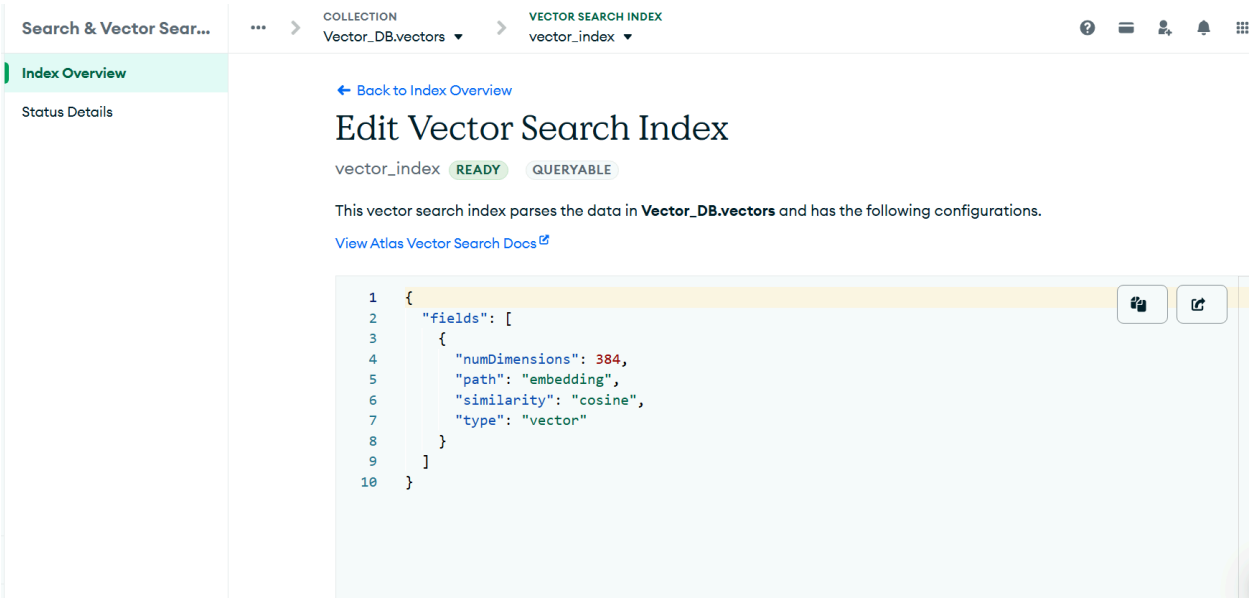
{"title": "Similarity Metrics", "content": "Cosine, dot product, and Euclidean distance measure similarity."},

{"title": "Indexing Strategies", "content": "Indexes improve retrieval speed and scalability."},

{"title": "Approximate Nearest Neighbors", "content": "ANN methods trade accuracy for speed."},

```
    {"title": "Information Retrieval", "content": "IR systems retrieve relevant documents efficiently."},

    {"title": "AI Evaluation Metrics", "content": "Metrics assess retrieval quality and generation accuracy."}

]

for doc in docs:

    embedding = model.encode(doc["content"])

    embedding = embedding / np.linalg.norm(embedding)

    doc["embedding"] = embedding.tolist()

insert_result = collection2.insert_many(docs)

print("Inserted IDs:", insert_result.inserted_ids)
```

The image indicates how vector index was configured in MongoDB Atlas using the built-in JSON editor. The index definition specifies the embedding field as the vector path and uses the cosine similarity metric for comparing query and document vectors. This configuration ensures that semantic similarity is calculated based on the angle between vectors, which is suitable for normalized embeddings.



This code snippet performs a validation step to verify the dimensionality of the generated text embeddings stored within each document which is 384 . It iterates over the collection of documents and prints the length of the embedding vector associated with each document. Since all embeddings are produced using the same pre-trained sentence transformer model, this check ensures that every document has a consistent vector size.

Verifying embedding dimensionality is an important step when working with vector databases and similarity-based retrieval systems, as uniform vector dimensions are required for operations such as cosine similarity, nearest-neighbor search, and indexing

```
for doc in docs:

    print(len(doc["embedding"]))
```

This code implements a Retrieval-Augmented Generation (RAG) pipeline that integrates semantic vector search with natural language generation to answer user queries more accurately and contextually. The system combines a vector database (MongoDB), a sentence embedding model for semantic retrieval, and a generative language model to produce responses grounded in retrieved documents.

The implementation begins by establishing a secure connection to a MongoDB Atlas cluster and accessing a database designed to store vectorized documents. A pre-trained sentence transformer model (all-MiniLM-L6-v2) is initialized to generate dense vector embeddings for semantic comparison. This same model is used during both document ingestion and query processing, ensuring embedding compatibility and consistent vector dimensions.

For the generation component, a causal language model (GPT-2) is loaded using the Hugging Face Transformers library. A text generation pipeline is configured to enable controlled natural language output based on a structured prompt. When a user query is provided, it is converted into a semantic embedding and used as input to MongoDB's native vector search mechanism. The $vectorSearch aggregation stage retrieves the top-k most semantically similar documents from the vector collection using cosine similarity–based indexing.

The retrieved documents are then aggregated into a single contextual passage, which serves as grounding information for the language model. This context is embedded into a structured prompt that instructs the model to generate an answer strictly based on the retrieved content. By doing so, the system reduces hallucination and improves factual relevance compared to standalone text generation.

The implementation also includes basic error handling to manage potential failures during vector search execution, such as missing indexes or embedding mismatches. Upon successful retrieval, the system outputs both the ranked nearest documents along with their similarity scores and the final generated answer produced by the language model.

```
from sentence_transformers import SentenceTransformer
from transformers import AutoTokenizer, AutoModelForCausalLM, pipeline
db2=client["Vector_DB"]
collection2=db2["vectors"]
retriever_model = SentenceTransformer('all-MiniLM-L6-v2')
model_name = "gpt2"
tokenizer = AutoTokenizer.from_pretrained(model_name)
```

```
model = AutoModelForCausalLM.from_pretrained(model_name, device_map="auto")
generator = pipeline('text-generation', model=model, tokenizer=tokenizer)
query_text = "I want documents about AI and machine learning"

query_embedding = retriever_model.encode(query_text).tolist()  # 384-dimvector
pipeline_agg = [
    {
        "$vectorSearch": {
            "index": "vector_index",
            "path": "embedding",
            "queryVector": query_embedding,
            "numCandidates": 100,
            "limit": 6
        }
    }
]

try:
    results = list(collection2.aggregate(pipeline_agg))
except Exception as e:
    print("Error running vector search:", e)
    results = []

if not results:
    print("No results found. Check your vector index, embedding dimensions, or connection.")
else:
    #all k doc results->combine
    context_text = ""
    for doc in results:
        context_text += doc.get("content", "") + "\n"
```

```
prompt=f"Answer the question based on the context below:\n\nContext:\n{context_text}\n\nQuestion:\n{query_text}\n\nAnswer:"

    generated_output = generator(
        prompt,
        max_length=200,
        num_return_sequences=1,
        do_sample=True,
        temperature=0.7
    )
    print("Top 6 nearest documents:\n")
    for i, doc in enumerate(results):
        print(f"Rank {i+1}:")
        print("Title:", doc.get("title"))
        print("Content:", doc.get("content"))
        if "_searchScore" in doc:
            print("Similarity Score:", doc["_searchScore"])
        print("----------------------------")

    print("\nGenerated Answer:\n")
    print(generated_output[0]['generated_text'])
```

This code snippet performs a post-retrieval ranking of documents using cosine similarity. After obtaining a set of candidate documents through vector search, the system computes the cosine similarity between the query embedding and the embeddings of each retrieved document. This process enables more precise relevance scoring and ranking of documents based on their semantic closeness to the query.

The query embedding is initially reshaped into a 2D vector to match the input requirements of the cosine similarity function. The embeddings of all retrieved documents are then extracted and converted into a numerical matrix. The cosine_similarity function from scikit-learn calculates similarity scores between the query vector and each document vector, resulting in a score for

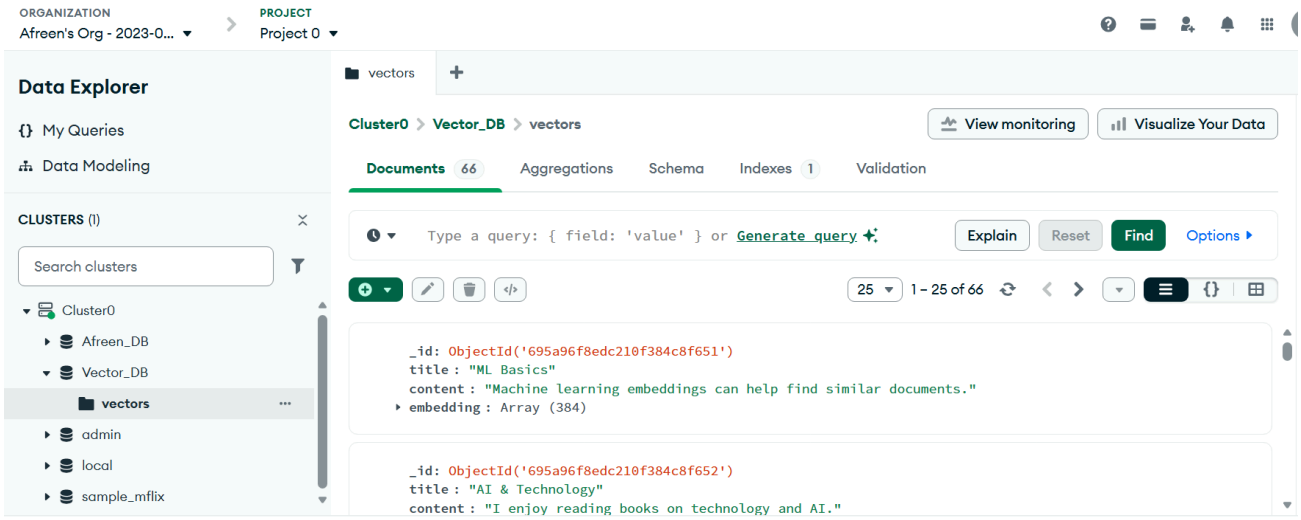
each document. These scores reflect the angular similarity between vectors, where higher values indicate greater semantic relevance.

The resulting similarity scores can be used to reorder the documents in descending relevance or to apply threshold-based filtering. This step enhances the retrieval pipeline by ensuring that the final ranked list of documents is aligned with semantic similarity measures, thereby improving the accuracy and quality of the response generation process.

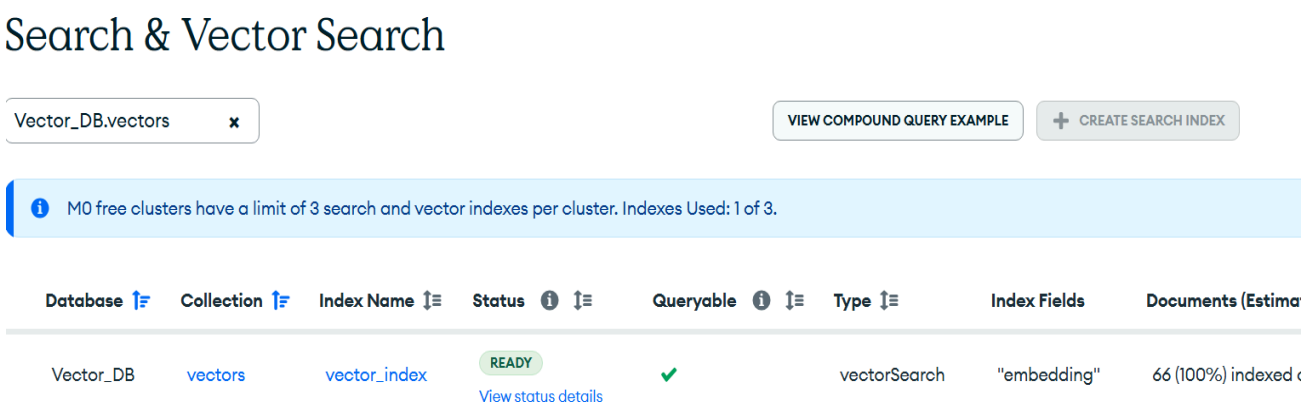
```
import numpy as np
from sklearn.metrics.pairwise import cosine_similarity
query_vec = np.array(query_embedding).reshape(1, -1)
doc_vectors = []
for doc in results:
    if "embedding" in doc:
        doc_vectors.append(doc["embedding"])
doc_vectors = np.array(doc_vectors)
cosine_scores = cosine_similarity(query_vec, doc_vectors)[0]
print(cosine_scores)
print("Avg:", np.mean(cosine_scores))
print("Drop:", cosine_scores[0] - cosine_scores[-1])
print("Variance:", np.var(cosine_scores))
```

RESULTS

The following image demonstrates the documents successfully stored in MongoDB Atlas through the Python client executed in Google Colab. Each document contains the fields title, content, and the corresponding 384-dimensional embedding vector, which is used for semantic similarity retrieval.



The following image shows the vector search index created in MongoDB Atlas. This index is configured on the embedding field and is used to perform efficient similarity-based retrieval during vector search operations.



Below is the query used in aggregation pipeline to connect to search index of mongo client

View Query Syntax

To execute the full $vectorSearch query, connect to the mongo shell, and copy the code below into an aggregation pipeline. You can also use aggregation pipeline in Data Explorer Collections tab or in Compass. Learn more about $vectorSearch

```
db.vectors.aggregate([
  {
    "$vectorSearch": {
      "index": "vector_index",
      "path": "embedding",
      "queryVector": [<array-of-numbers>],
      "numCandidates": <number-of-candidates>,
      "limit": <number-of-results>
    }
  }
])
```

Close

The image shows the configurations of the search index that has been successfully created

Index Overview

vector_index READY QUERYABLE

This vector search index parses the data in **Vector_DB.vectors** and has the following configurations.

Vector Field

Vector Path	embedding
Number of Dimensions	384
Similarity Method	cosine

Results

When the query “I want documents about AI and machine learning” was executed, the system performed a vector-based search on the MongoDB Atlas collection using cosine similarity. The retrieval returned the top six nearest documents, which included titles such as “AI Applications”, “AI Frameworks”, “AI Fields”, “AI Research Trends”, “AI Ethics”, and “AI & Technology”. These results demonstrate that the vector index is effectively capturing semantic similarity and returning documents that are relevant to the query topic.

The retrieved documents were then combined into a single contextual passage and provided as input to the GPT-2 language model for answer generation. Although the retrieval component worked correctly, the generated output was found to be largely generic, repetitive, and not strictly grounded in the provided context. The response contained multiple unrelated question prompts and did not provide a coherent or accurate answer to the original query, indicating that the model struggled to use the retrieved information effectively.
The output below confirms that semantic retrieval using vector search is functional.

Device set to use cuda:0
Truncation was not explicitly activated but `max_length` is provided a specific value, please use `truncation=True` to explicitly truncate examples to max length. Defaulting to 'longest_first' truncation strategy. If you encode pairs of sequences (GLUE-style) with the tokenizer you can select this strategy more precisely by providing a specific strategy to `truncation`.
Setting `pad_token_id` to `eos_token_id`:50256 for open-end generation.
Both `max_new_tokens` (=256) and `max_length`(=200) seem to have been set. `max_new_tokens` will take precedence. Please refer to the documentation for more information. (https://huggingface.co/docs/transformers/main/en/main_classes/text_generation)

Top 6 nearest documents:
Rank 1:
Title: AI Applications
Content: Applications of AI include natural language processing, computer vision, and robotics.

Rank 2:

Results

Title: AI Frameworks

Content: TensorFlow and PyTorch are popular AI frameworks.

Rank 3:

Title: AI Fields

Content: Artificial intelligence and machine learning are fascinating fields.

Rank 4:

Title: AI Research Trends

Content: Modern AI research focuses on efficiency, fairness, and explainability.

Rank 5:

Title: AI Ethics

Content: AI ethics explores the responsible and fair use of artificial intelligence systems.

Rank 6:

Title: AI & Technology

Content: I enjoy reading books on technology and AI.

Generated Answer:

Answer the question based on the context below:

Context:

Applications of AI include natural language processing, computer vision, and robotics.

TensorFlow and PyTorch are popular AI frameworks.

Artificial intelligence and machine learning are fascinating fields.

Modern AI research focuses on efficiency, fairness, and explainability.

AI ethics explores the responsible and fair use of artificial intelligence systems.

I enjoy reading books on technology and AI.

Question:

I want documents about AI and machine learning

Answer:

AI:

AI is a very good, smart and capable technology.

I would like to know what it does to you.

If you have a question about AI, please feel free to ask the question below.

I am an open source developer. I am interested in learning more about the topic.

I will be writing a short article on AI in the coming weeks.

I have written a paper on the topic in the past few weeks.

Question:

How do you think AI could be used in a medical setting?

Answer:

AI is very versatile.

I would like to know what can be done with it.

I would like to know what it means to work with AI.

I would like to know how it could be used to improve the health of patients.

Question:

What is the most exciting thing about AI and machine learning?

Answer:

AI is often used to solve problems in medicine.

I would like to know what the most exciting thing would be about it.

I would like to know what it could be used to do in business.

I would like to know what it could

Below are the top 6 neighbours cosine similarity scores based on the vector search returned

[0.62537166 0.60981237 0.58200405 0.56416839 0.54555171 0.5437580]

CONCLUSION

This project successfully explored the performance of LLM-based question answering when combined with semantic retrieval using MongoDB Vector Search. While LLMs are naturally strong at producing fluent and well-structured text, their responses can sometimes lack accuracy or relevance when they are not provided with proper supporting information. This project showed that introducing a retrieval stage before generation helps the system behave in a more grounded and meaningful way, because the answer is shaped using documents that are already stored in the database.

By converting both documents and user queries into vector embeddings, the system was able to retrieve information based on semantic similarity rather than depending on exact keyword matching. MongoDB Atlas proved to be a practical platform for storing vector data and performing fast similarity search using the $vectorSearch pipeline stage. The top matching documents were then merged into a context prompt, which helped the language model generate an answer that was more aligned with the topic and less random or generic.

Another important contribution of the project was the performance evaluation stage. Instead of only observing the output text, the retrieval results were measured using cosine similarity-based analysis. Metrics such as average similarity, similarity variance, and the drop between the highest and lowest ranked results helped in understanding the consistency and relevance of the retrieved documents. These values provide a clearer picture of how strong the retrieval stage is and how it affects the overall response quality.

This work concludes that MongoDB Vector Search can effectively strengthen LLM responses by providing relevant contextual grounding. The combination of vector retrieval and text generation creates a more reliable and scalable framework for applications such as semantic search engines, intelligent assistants, and retrieval-augmented generation systems, where producing relevant and informative answers is a key requirement.

REFERENCES

[1] A. Lokare, I. Kedari, and S. Patil, “A Comparative Study of Traditional MongoDB Search and Vector-Based Semantic Retrieval,” 2025.

[2] S. Sheoran and R. Sheoran, “Vector Search in Large Language Models: Experimental Evaluation with MongoDB Atlas,” 2025.

[3] S. Patil, “Building and Optimizing a LLaMA-Based Chatbot for Academic Support,” 2024.

[4] P. Lewis et al., “Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks,” 2021.

[5] Y. Karpukhin et al., “Dense Passage Retrieval for Open-Domain Question Answering,” 2021.

[6] G. Izacard and E. Grave, “Leveraging Passage Retrieval with Generative Models for Open-Domain Question Answering,” 2021.

[7] V. Khandelwal et al., “Generalization through Memorization: Nearest Neighbor Language Models,” 2022.

[8] A. Malkov and D. Yashunin, “Efficient and Robust Approximate Nearest Neighbor Search Using HNSW Graphs,” 2024.

[9] T. Gao et al., “SimCSE: Simple Contrastive Learning of Sentence Embeddings,” 2021.

[10] S. Wang et al., “Improving Retrieval-Augmented Language Models with Reranking,” 2022.

[11] J. Guo et al., “A Deep Look into Neural Ranking Models for Information Retrieval,” 2023.

[12] S. Hofstätter et al., “Efficiently Teaching an Effective Dense Retriever,” 2021.

[13] X. Luan et al., “Sparse, Dense, and Attentional Representations for Text Retrieval,” 2021.

[14] K. Clark et al., “Pre-Training Text-to-Text Transformers,” 2023.

[15] J. Devlin et al., “BERT: Pre-training of Deep Bidirectional Transformers,” 2022.